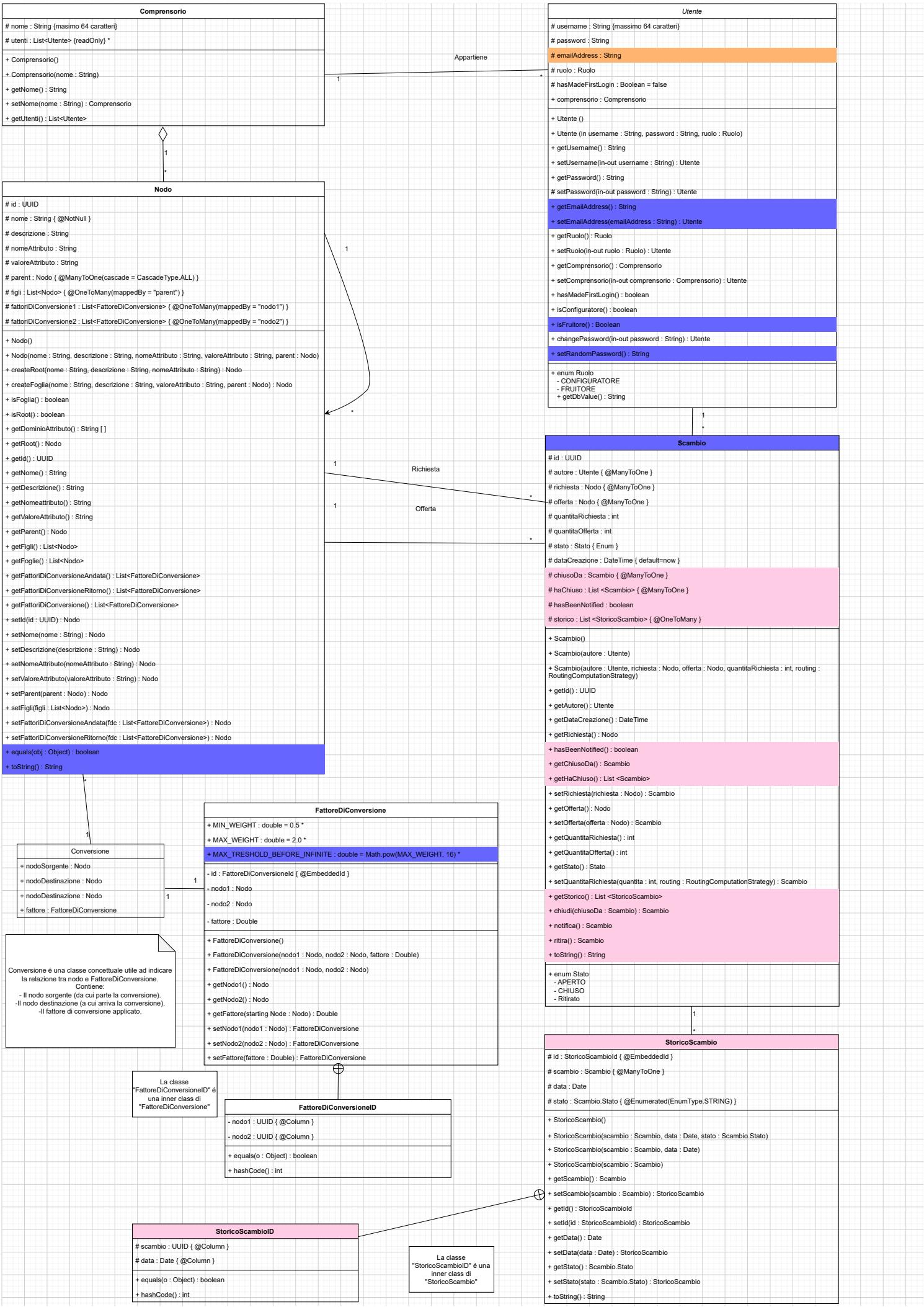

Comprensorio
nome : String {masimo 64 caratteri}
utenti : List<Utente> {readOnly} *
+ Comprensorio()
+ Comprensorio(nome : String)
+ getNome() : String
+ setNome(nome : String) : Comprensorio
+ getUtenti() : List<Utente>
Appartiene
1
*
Utente
username : String {massimo 64 caratteri}
password : String
emailAddress : String
ruolo : Ruolo
hasMadeFirstLogin : Boolean = false
+ comprensorio : Comprensorio
+ Utente ()
+ Utente (in username : String, password : String, ruolo : Ruolo)
+ getUsername() : String
+ setUsername(in-out username : String) : Utente
+ getPassword() : String
setPassword(in-out password : String) : Utente
+ getEmailAddress() : String
+ setEmailAddress(emailAddress : String) : Utente
+ getRuolo() : Ruolo
+ setRuolo(in-out ruolo : Ruolo) : Utente
+ getComprensorio() : Comprensorio
+ setComprensorio(in-out comprensorio : Comprensorio) : Utente
+ hasMadeFirstLogin() : boolean
+ isConfiguratore() : boolean
+ isFruitore() : Boolean
+ changePassword(in-out password : String) : Utente
+ setRandomPassword() : String
+ enum Ruolo
- CONFIGURATORE
- FRUITORE
+ getDbValue() : String
1
*
Nodo
id : UUID
nome : String { @NotNull }
descrizione : String
nomeAttributo : String
valoreAttributo : String
parent : Nodo { @ManyToOne(cascade = CascadeType.ALL) }
figli : List<Nodo> { @OneToMany(mappedBy = "parent") }
fattoriDiConversione1 : List<FattoreDiConversione> { @OneToMany(mappedBy = "nodo1") }
fattoriDiConversione2 : List<FattoreDiConversione> { @OneToMany(mappedBy = "nodo2") }
+ Nodo()
+ Nodo(nome : String, descrizione : String, nomeAttributo : String, valoreAttributo : String, parent : Nodo)
+ createRoot(nome : String, descrizione : String, nomeAttributo : String) : Nodo
+ createFoglia(nome : String, descrizione : String, valoreAttributo : String, parent : Nodo) : Nodo
+ isFoglia() : boolean
+ isRoot() : boolean
+ getDominioAttributo() : String [ ]
+ getRoot() : Nodo
+ getId() : UUID
+ getNome() : String
+ getDescrizione() : String
+ getNomeattributo() : String
+ getValoreAttributo() : String
+ getParent() : Nodo
+ getFigli() : List<Nodo>
+ getFoglie() : List<Nodo>
+ getFattoriDiConversioneAndata() : List<FattoreDiConversione>
+ getFattoriDiConversioneRitorno() : List<FattoreDiConversione>
+ getFattoriDiConversione() : List<FattoreDiConversione>
+ setId(id : UUID) : Nodo
+ setNome(nome : String) : Nodo
+ setDescrizione(descrizione : String) : Nodo
+ setNomeAttributo(nomeAttributo : String) : Nodo
+ setValoreAttributo(valoreAttributo : String) : Nodo
+ setParent(parent : Nodo) : Nodo
+ setFigli(figli : List<Nodo>) : Nodo
+ setFattoriDiConversioneAndata(fdc : List<FattoreDiConversione>) : Nodo
+ setFattoriDiConversioneRitorno(fdc : List<FattoreDiConversione>) : Nodo
+ equals(obj : Object) : boolean
+ toString() : String
1
*
Richiesta
Offerta
Scambio
id : UUID
autore : Utente { @ManyToOne }
richiesta : Nodo { @ManyToOne }
offerta : Nodo { @ManyToOne }
quantitaRichiesta : int
quantitaOfferta : int
stato : Stato { Enum }
dataCreazione : DateTime { default=now }
chiusoDa : Scambio { @ManyToOne }
haChiuso : List <Scambio> { @ManyToOne }
hasBeenNotified : boolean
storico : List <StoricoScambio> { @OneToMany }
+ Scambio()
+ Scambio(autore : Utente)
+ Scambio(autore : Utente, richiesta : Nodo, offerta : Nodo, quantitaRichiesta : int, routing : RoutingComputationStrategy)
+ getId() : UUID
+ getAutore() : Utente
+ getDataCreazione() : DateTime
+ getRichiesta() : Nodo
+ hasBeenNotified() : boolean
+ getChiusoDa() : Scambio
+ getHaChiuso() : List <Scambio>
+ setRichiesta(richiesta : Nodo) : Scambio
+ getOfferta() : Nodo
+ setOfferta(offerta : Nodo) : Scambio
+ getQuantitaRichiesta() : int
+ getQuantitaOfferta() : int
+ getStato() : Stato
+ setQuantitaRichiesta(quantita : int, routing : RoutingComputationStrategy) : Scambio
+ getStorico() : List <StoricoScambio>
+ chiudi(chiusoDa : Scambio) : Scambio
+ notifica() : Scambio
+ ritira() : Scambio
+ toString() : String
+ enum Stato
- APERTO
- CHIUSO
- Ritirato
1
*
Conversione
+ nodoSorgente : Nodo
+ nodoDestinazione : Nodo
+ nodoDestinazione : Nodo
+ fattore : FattoreDiConversione
1
1
FattoreDiConversione
+ MIN_WEIGHT : double = 0.5 *
+ MAX_WEIGHT : double = 2.0 *
+ MAX_TRESHOLD_BEFORE_INFINITE : double = Math.pow(MAX_WEIGHT, 16) *
- id : FattoreDiConversioneId { @EmbeddedId }
- nodo1 : Nodo
- nodo2 : Nodo
- fattore : Double
+ FattoreDiConversione()
+ FattoreDiConversione(nodo1 : Nodo, nodo2 : Nodo, fattore : Double)
+ FattoreDiConversione(nodo1 : Nodo, nodo2 : Nodo)
+ getNodo1() : Nodo
+ getNodo2() : Nodo
+ getFattore(starting Node : Nodo) : Double
+ setNodo1(nodo1 : Nodo) : FattoreDiConversione
+ setNodo2(nodo2 : Nodo) : FattoreDiConversione
+ setFattore(fattore : Double) : FattoreDiConversione
Conversione é una classe concettuale utile ad indicare la relazione tra nodo e FattoreDiConversione.
Contiene:
- Il nodo sorgente (da cui parte la conversione).
-Il nodo destinazione (a cui arriva la conversione).
-Il fattore di conversione applicato.
La classe "FattoreDiConversioneID" é una inner class di "FattoreDiConversione"
FattoreDiConversioneID
- nodo1 : UUID { @Column }
- nodo2 : UUID { @Column }
+ equals(o : Object) : boolean
+ hashCode() : int
StoricoScambio
id : StoricoScambioId { @EmbeddedId }
scambio : Scambio { @ManyToOne }
data : Date
stato : Scambio.Stato { @Enumerated(EnumType.STRING) }
+ StoricoScambio()
+ StoricoScambio(scambio : Scambio, data : Date, stato : Scambio.Stato)
+ StoricoScambio(scambio : Scambio, data : Date)
+ StoricoScambio(scambio : Scambio)
+ getScambio() : Scambio
+ setScambio(scambio : Scambio) : StoricoScambio
+ getId() : StoricoScambioId
+ setId(id : StoricoScambioId) : StoricoScambio
+ getData() : Date
+ setData(data : Date) : StoricoScambio
+ getStato() : Scambio.Stato
+ setStato(stato : Scambio.Stato) : StoricoScambio
+ toString() : String
StoricoScambioID
scambio : UUID { @Column }
data : Date { @Column }
+ equals(o : Object) : boolean
+ hashCode() : int
La classe "StoricoScambioID" é una inner class di "StoricoScambio"